Bitácora técnica — AlojaME

> Documento de referencia técnica del proyecto: arquitectura, reglas de negocio y el historial de errores encontrados y resueltos (con su causa y su solución), para no volver a cometerlos. Las decisiones de diseño del frontend están en *Documentacion_Frontend_AlojaME.pdf*.

Stack

- Java 21, Spring Boot 3.4.3, MySQL/MariaDB, JPA/Hibernate, Spring Security + JJWT 0.12.6
- Cloudinary (imágenes), Springdoc/Swagger, Lombok
- Paquete base: `com.eloyroldan.tp_final_alojamiento.backend`
- Base de datos: `testdb`
- `spring.jpa.hibernate.ddl-auto=update` — las altas incrementales (tablas/columnas nuevas) las genera Hibernate solo. Excepción: columnas `GENERATED ALWAYS AS` (ej. `reserva.precio_total`) y `CHECK` constraints NO las puede generar Hibernate desde cero — ver sección de gotchas más abajo.
- `spring.jpa.open-in-view=false` → nunca se devuelven entidades JPA directas desde un Controller. Todo pasa por DTOs de Response armados dentro de métodos `@Transactional`.

Regla de arquitectura

Un Service solo puede llamar a su propio Repository — nunca al Repository de otra entidad. Si necesita datos de otra tabla, llama al Service dueño de esa tabla, no a su Repository directo.

Los ciclos de dependencia entre Services (inevitables en algunos casos) se rompen con `@Lazy` del lado que menos lo necesita:

- `AlojamientoService` ↔ `ReservaService` (`@Lazy` en `AlojamientoService`)
- `PagoService` ↔ `ReservaService` (`@Lazy` en `PagoService`, ya que `ReservaService` depende de `PagoService` en casi todas sus transiciones de estado, así que esa dirección va eager)
- `UsuarioService` ↔ `VerificacionService` (`@Lazy` en `UsuarioService`)
- Patrón similar para `TipoAlojamientoService` ↔ `AlojamientoService`, `DireccionService` ↔ `AlojamientoService`, `ServicioService` ↔ `ServicioxAlojamientoService`

Antes de agregar cualquier método nuevo a un Service, verificar que no esté tocando un Repository ajeno. Hay un patrón de auditoría automática usado varias veces en el proyecto: grepear `private final \w+Repository` en cada Service y comparar contra el nombre del propio Service.

Politicas de cancelacion (Strategy) y pasarela de pago (DIP)

`PagoService.ejecutarCancelacion` NO tiene la matematica de reembolso/retencion de cada `MOTIVOCANCELACION` adentro: cada motivo (salvo `VENCIMIENTO_PAGO`, que se resuelve antes y nunca llega a este punto) tiene su propia clase en `Service/Cancelacion/` que implementa `PoliticaCancelacion` (`ReembolsoTotalPolicy`, `RetencionTardiaPolicy`, `ReembolsoManualAdminPolicy`). `PagoService` las indexa por motivo en un `Map` (`@PostConstruct`) e

inyecta `List<PoliticaCancelacion>` — agregar un motivo nuevo es agregar una clase en ese paquete, nunca volver a meter un `switch` en `PagoService`.

`PagoService` y las `PoliticaCancelacion` dependen de la interfaz `PasarelaDePagoService` (`crearPreferencia` / `consultarPago` / `reembolsar` / `crearPayout`), no de `MercadoPagoService` directo — hoy hay una sola implementación, pero cambiar de proveedor de pagos es agregar una implementación nueva de esa interfaz, no reescribir el negocio.

Redondeo monetario compartido: `Service/MontoUtil.redondear()` — no repetir `setScale(2, HALF_UP)` suelto en un Service o policy nuevo.

Estructura de paquetes

```
backend/
  Config/          SecurityConfig, JwtAuthFilter, CustomUserDetailsService,
CloudinaryConfig
  Converter/       AttributeConverters para enums (la BD usa minúsculas, Java mayúsculas)
  DTO/             Request DTOs + Response/ (DTOs de salida)
  Enum/            ESTADOPAGO, ESTADORESERVA, TIPOUSUARIO, MOTIVOCANCELACION
  Exception/       GlobalExceptionHandler + excepciones custom
  Model/           Entidades JPA (Alojamiento, Reserva, Pago, Usuario, Review, etc.)
  Repository/      Uno por entidad, misma subestructura que Model/
  RestController/  Endpoints REST
  Service/         Lógica de negocio — un Service por entidad, más algunos orquestadores
(EstadisticasController llama a varios Services sin ser dueño de ninguna tabla)
  Validation/       Bean Validation custom (ej. DominioEmailValido)
db/                 SQL sueltos (schema_snapshot.sql = foto completa del esquema actual,
completar_indices_y_checks.sql = lo único que Hibernate no puede generar solo)
```

Modelo de dominio — máquinas de estado

Reserva

```
SOLICITADA → CONFIRMADA → FINALIZADA
           ↘ CANCELADA ↙ (desde SOLICITADA o CONFIRMADA)
```

- Cliente crea en `SOLICITADA`.
- Anfitrión (o admin) confirma → `CONFIRMADA`, arranca el plazo de pago de 48hs (`fecha_confirmacion`), se crea el `Pago` + preferencia de MercadoPago.
- Se cancela sola si no se paga en 48hs (job `@Scheduled` en `PagoService.cancelarPagosVencidos`).
- `FINALIZADA` cuando pasó `fecha_fin` y el anfitrión (o admin) la marca — dispara el Payout al anfitrión.
- Condición de carrera resuelta: crear/reprogramar una reserva usa `AlojamientoService.obtenerPorIdConLock()` (bloqueo pesimista `PESSIMISTIC_WRITE`), no `obtenerPorId()`, para que dos solicitudes simultáneas sobre el mismo alojamiento no pasen juntas el chequeo de solapamiento de fechas.

Pago (Modelo B: la plataforma retiene y liquida)

```
PENDIENTE → PAGADO → (REEMBOLSADO_TOTAL | REEMBOLSADO_PARCIAL, si se cancela)
PENDIENTE → VENCIDO (si pasan 48hs sin pagar)
```

- Gross-up: el anfitrión define `monto_base` (lo que quiere recibir neto). El cliente paga `monto_base + comisión_plataforma (5%) + comisión_mercadopago_estimada (6%, configurable)`.
- El anfitrión nunca interactúa con MercadoPago — solo carga un alias/CBU/CVU (`Usuario.aliasCobro`) validado con dígito verificador BCRA + identificación de banco (`AliasCobroService`), más una consulta MX por DNS de respaldo para dominios de email no comunes (`DominioEmailValidoValidator`).
- Política de cancelación (`ReservaService.determinarMotivo`):
  - Anfitrión cancela → reembolso 100% al cliente, siempre.
  - Cliente cancela con ≥48hs (2 días) de anticipación → reembolso 100%.
  - Cliente cancela con <48hs → reembolso 65%, el anfitrión recibe el 35% restante vía Payout (aunque la estadía nunca ocurrió).
  - Vencimiento de pago (nunca se pagó) → sin plata movida, `VENCIDO`.
  - Admin → reembolso a medida (`monto <= monto_base`, validado).
- Los fees (comisión plataforma + MercadoPago) nunca se devuelven, en ningún escenario.
- El Payout al anfitrión se dispara en 2 momentos distintos: al `FINALIZADA` (100%) o en cancelación tardía (35%).

Seguridad — matriz de roles

3 roles: `ADMIN`, `ANFITRION`, `CLIENTE`. Definida en `SecurityConfig` (JWT stateless, sin sesión). El chequeo de "es dueño del recurso" (ej: un cliente solo cancela SU reserva) no vive en `SecurityConfig` — eso lo resuelve cada Service/Controller puntual comparando IDs.

Registro y verificación

Usuario se registra con `email_verificado=false`, bloqueo total de login hasta confirmar vía token de un solo uso (24hs, tabla `token_verificacion`) enviado por Brevo. Admin se crea ya verificado. Validación de dominio de email híbrida: whitelist rápida de proveedores comunes + consulta MX por DNS de respaldo para el resto (`DominioEmailValidoValidator`).

Errores resueltos del backend

1. `reserva.precio_total` es una columna `GENERATED ALWAYS AS (precio_noche_pactado * DATEDIFF(fecha_fin, fecha_inicio))`, pero Hibernate NUNCA la puede recrear desde cero (no tiene anotación para eso) y en MariaDB 10.4 un `ALTER TABLE` sobre OTRA columna puede hacer que la tabla se reconstruya y esa columna quede en null. Por eso `PagoService.calcularMontoBase()` recalcula noches×precio en Java y jamás confía en `Reserva.getPrecioTotal()`. No volver a depender de esa columna para nada crítico.

2. `JwtAuthFilter` corre antes del `DispatcherServlet`, fuera del alcance de `GlobalExceptionHandler`. Un JWT vencido o manipulado tira `JwtException` / `IllegalArgumentException` sin capturar si no se blinda ahí mismo con

try/catch — ya está resuelto, pero cualquier cambio a ese filtro tiene que mantener ese try/catch.

3. Los enums se guardan en minúscula en la BD, en mayúscula en Java. Se resuelve con `AttributeConverter` (no `@Enumerated`), no volver a usar `@Enumerated` directo.

4. FK sin cascada entre `pago` e `reserva` (`pago.id_reserva → reserva.id_reserva`, sin `ON DELETE CASCADE`). Cualquier borrado de una `Reserva` (individual o en cascada desde `Alojamiento`) tiene que borrar el `Pago` asociado ANTES (`PagoService.borrarPorReserva` / `borrarPorAlojamiento`), o la FK lo rechaza.

5. N+1 queries: los listados de `Alojamiento`, `Reserva` y los resúmenes de `Estadisticas` usan `JOIN FETCH` o consultas "en lote" (`Map<Long, X>` con una sola query agrupada) en vez de una query por fila. Si se agrega un nuevo listado, replicar ese patrón, no iterar y consultar de a uno.

6. `/actuator/prometheus` requiere la dependencia `micrometer-registry-prometheus` además de `spring-boot-starter-actuator` — sin eso, el endpoint no se registra aunque esté en `management.endpoints.web.exposure.include`. **Ya agregada** en el `pom.xml` (se detectó que faltaba pese a estar documentada: el endpoint devolvía 404).

7. Índices y `CHECK` constraints se declaran vía anotaciones JPA (`@Table(indexes=...)`, `@Check` de Hibernate) en los Models — así Hibernate los genera solo en cualquier base nueva. `db/completar_indices_y_checks.sql` es el fallback manual, de una sola vez, solo para bases que ya existían antes de agregar esas anotaciones.

8. `JwtAuthFilter` validaba la firma/vencimiento del token pero no `userDetails.isEnabled()`: una cuenta recién desactivada (`Usuario.activo=false`) seguía autenticando con un JWT emitido antes de la baja hasta por 24hs. **Ya resuelto** — el filtro ahora exige `isTokenValid(...) && userDetails.isEnabled()`.

9. `confirmar` / `cancelar` / `cancelarComoAdmin` / `finalizar` de `ReservaService` leían la `Reserva` con `obtenerPorId` (sin lock): dos transiciones de estado concurrentes sobre la MISMA reserva podían pasar juntas el chequeo de estado y disparar el Payout/reembolso dos veces. **Ya resuelto** — usan `obtenerPorIdConLock` (`ReservaRepository.findByIdConLock`, `PESSIMISTIC_WRITE`), mismo patrón que `AlojamientoService.obtenerPorIdConLock`.

10. `application.properties` tenía credenciales reales hardcodeadas (DB, Cloudinary, JWT secret, Brevo, MercadoPago) — algunas como default de `${VAR:...}`, otras directo. **Ya resuelto**: las 6 dependen de su variable de entorno sin ningún default — la app falla al arrancar si falta alguna, en vez de arrancar con un secreto comprometido. En producción se usan credenciales nuevas (Cloudinary, Brevo, MercadoPago y un `jwt.secret` propio), nunca las de desarrollo.

11. CORS tenía `setAllowedOriginPatterns(List.of("*"))` combinado con `allowCredentials(true)`: cualquier sitio externo podía mandar requests autenticados (JWT) contra esta API desde el browser del usuario. **Ya resuelto** — `cors.allowed-origins` (lista explícita, por defecto solo `app.frontend-url`).

12. `ReservaService.cancelar` / `cancelarComoAdmin` llamaban siempre a `PagoService.ejecutarCancelacion`, que asume que ya existe un `Pago` (solo se crea en

`confirmar()`). Cancelar una reserva todavía `SOLICITADA` (transición válida según el diagrama de arriba) tiraba `IdNoEncontradoException` ("Esta reserva todavía no tiene un pago asociado") en vez de cancelarla — reproducido en vivo con `curl` contra una MariaDB real antes de corregirlo. La transacción rollbackeaba bien (no corrompía datos), pero la operación no funcionaba. **Ya resuelto** — ambos métodos solo llaman a `PagoService.ejecutarCancelacion` cuando la reserva ya pasó por `CONFIRMADA`.

13. **SecurityConfig** abre a lectura pública (`permitAll`, acotado con `HttpMethod.GET`) los endpoints de solo lectura para navegar el catálogo sin cuenta: listar/mostrar `Alojamiento`, sus servicios, `Galeria`, `Review` por alojamiento, y los catálogos `TipoAlojamiento`/`Servicio`. Decisión explícita del equipo (modelo tipo Airbnb: se navega libre, el login solo hace falta para publicar/reservar/cualquier escritura) — antes de esto, **ningún** endpoint de `Alojamiento` era público, ni siquiera listar. Donde un mismo path sirve lectura y escritura (ej. `/Alojamiento/{id}/servicios`: GET lista, POST agrega) el `permitAll` está acotado por método HTTP, nunca al path entero — no volver a hacerlo público sin ese acotamiento.

14. **`MercadoPagoService.crearPayout` (API de Payouts) devuelve `400 Bad Request: "Invalid signature"`** al probarlo con las credenciales TEST reales de la app de developers usada en este proyecto — confirmado en vivo llamando a `ReservaService.finalizar()` de punta a punta (reserva real, confirmada, pagada y con webhook procesado, todo contra la API real). `crearPreferencia`, `consultarPago` y el webhook **sí** funcionan perfecto con las mismas credenciales; es específicamente el endpoint `/v1/payouts` el que rechaza la llamada. Ya estaba anticipado en el comentario original de `crearPayout` ("hay que confirmarlo contra el panel de developers... misma limitación de sandbox") — la API de Payouts necesita habilitación extra (cuenta marketplace/collector) que esta app de prueba no tiene. **No es un bug de este código.** Si se vuelve a probar `finalizar()`/el Payout de cancelación tardía con credenciales nuevas, probablemente haga falta habilitar Payouts específicamente en el panel de MercadoPago antes de que funcione.

15. **`TipoAlojamientoController`/`ServicioController` solo tenían `listar`/`reemplazar`/`eliminar`, sin alta ni edición en el lugar** — detectado al construir el Lote 4 del frontend (SCRUM-43/44 piden explícitamente "alta, edición y baja" del catálogo). Se agregaron `POST /crear` y `PUT /{id}` (`NombreCatalogoDTO { nombre }`) a ambos controllers, con los métodos `crear`/`renombrar` correspondientes en `TipoAlojamientoService`/`ServicioService` (mismo criterio de normalización que `obtenerOcrear`, pero sin reutilizar en silencio: `crear` tira error si ya existe un nombre igual, `renombrar` tira error si el nuevo nombre colisiona con OTRA fila — ahí corresponde `reemplazar`, no `renombrar`). `SecurityConfig` actualizado: `POST /TipoAlojamiento/crear` y `POST /Servicio/crear` van con el resto de los matchers `hasRole("ADMIN")` de cada catálogo; `PUT /TipoAlojamiento/*` y `PUT /Servicio/*` son matchers nuevos, acotados por `HttpMethod.PUT`, también `ADMIN`. Verificado en vivo: alta, edición, y rechazo con mensaje claro al intentar crear/renombrar con un nombre que ya existe.

16. **`POST /auth/reenviar-verificacion` era un oráculo de enumeración de usuarios** — detectado en la auditoría de lógica de negocio del Lote 4.2. Antes de la correción, `VerificacionService.reenviarVerificacion` tiraba una excepción DISTINTA segun el caso: `UnautorizedExeption("Usuario no encontrado: <email>")` si el email no existía, `ParametroInvalidoException("Esta cuenta ya fue verificada.")` si existía y ya estaba verificado, o éxito si existía y no estaba verificado — las 3 respuestas son distintas y

públicas (sin login), asi que cualquiera podía confirmar, probando emails al voleo, cuáles están registrados en la plataforma y cuáles ya verificaron su cuenta. El login (`/auth/login`) NO tiene este problema: Spring Security ya normaliza `UsernameNotFoundException` a `BadCredentialsException` (`hideUserNotFoundExceptions`, default `true`), asi que usuario inexistente y contraseña incorrecta devuelven el mismo mensaje — confirmado en vivo con curl. **Ya resuelto** — se agregó `UsuarioService.buscarPorEmail` (devuelve `Optional`, no tira excepción) y `reenviarVerificacion` ahora SIEMPRE responde igual sin importar el caso (si el email no existe, o ya está verificado, simplemente no hace nada; solo genera y envía un token nuevo cuando corresponde). El Controller nunca ve la diferencia.

17. **Cambiar el propio email en "Mi perfil" deslogueaba a mitad de sesión, sin ningún error visible** — detectado en la auditoría de lógica de negocio del Lote 4.2, reproducido en vivo con Playwright contra el backend real. Causa: `CustomUserDetailsService` usa siempre el **email** como `username` interno del `UserDetails`, y `JwtService.generateToken` firma el JWT con ese username como `subject` — asi que el JWT emitido al loguearse tiene el email VIEJO. `JwtAuthFilter` resuelve el usuario en CADA request autenticado llamando a `userDetailsService.loadUserByUsername(subjectDelToken)`; en cuanto `UsuarioService.actualizar` pisa el email en la fila de `usuario`, esa búsqueda por el email viejo deja de encontrar a nadie y el filtro simplemente no autentica el request (no tira ningún error visible: la próxima request autenticada se trata como anónima y termina en 401/redirect a `/login`). **Ya resuelto** — `UsuarioController.actualizar` detecta si el email cambió y, si cambió, reemite un JWT nuevo (con el email nuevo como subject) y lo devuelve en el header de respuesta `X-New-Token`; `CorsConfiguration` necesitó `setExposedHeaders(List.of("X-New-Token"))` porque sin eso el browser descarta el header antes de que el JS del front lo vea (`setAllowedHeaders` es para headers de REQUEST, no de response). Del lado del frontend, `UsuarioService.actualizar` pide la respuesta con `observe:'response'`, lee ese header, y si está presente llama a `AuthService.actualizarToken(...)` (método nuevo, wrapper público de `guardarSesion`) para reemplazar el JWT guardado de forma transparente — el usuario nunca se entera de que hubo un token nuevo. Verificado en vivo: cambiar el email ya no corta la sesión, la navegación siguiente (`/mis-reservas`) sigue autenticada. Si se agrega otro campo que participe de la identidad del `UserDetails` (hoy solo el email) y se vuelve editable, aplicar el mismo patrón.

18. **No hay rate limiting / bloqueo por intentos fallidos en `/auth/login`** — detectado en la auditoría de lógica de negocio del Lote 4.2. Nada frena hoy un ataque de fuerza bruta probando contraseñas contra un `identificador` conocido (no hay `Bucket4j` ni librería de rate limiting en el `pom.xml`, ni ningún contador de intentos fallidos). **Decisión explícita del equipo: no implementarlo** — queda documentado como gap conocido, no bloqueante para el alcance de este TP universitario. Si se retoma: un filtro liviano en memoria (ej. 5 intentos fallidos por `identificador`+IP en una ventana de 15 min, bloqueo temporal) alcanza sin agregar dependencias nuevas.

Errores resueltos del frontend

1. **`NG0200: Circular dependency detected for _AuthService`** al recargar la página con sesión activa (`AuthService` es `providedIn:'root'`). Causa: el constructor de `AuthService` llamaba `cargarPerfil()` (un `GET /Usuario/me`) de forma sincrónica cuando ya había un token

guardado; esa request pasa por `authInterceptor`, que hace `inject(AuthService)` — y como el constructor todavía no había terminado de devolver, Angular lo detecta como dependencia circular contra sí mismo. El error ni siquiera llega a salir a la red (se puede confirmar con `page.route()` de Playwright: la request nunca se dispara), cae directo en el `catchError` de `cargarPerfil()`, que llama a `cerrarSesion()` — **el usuario pierde la sesión en cada F5**, silenciosamente, sin ningún error visible en la UI. Reproducido y diagnosticado en vivo con Playwright contra el backend real (instrumentando `Storage.prototype.setItem/removeItem` para loguear el stack trace exacto que disparaba el `removeItem`). **Ya resuelto** — la llamada inicial a `cargarPerfil()` se difiere con `queueMicrotask(...)`, así corre después de que el constructor ya devolvió y el injector tiene la instancia completamente registrada. Si se agrega otro servicio `providedIn:'root'` cuyo constructor dispare una request HTTP de entrada (no a pedido de un componente), aplicar el mismo patrón.
2. **Metodología para probar el flujo de pago real (MercadoPago) contra el sandbox de MercadoPago**, usada en el Lote 2 y reusable en el Lote 3: (a) la API de MercadoPago (`api.mercadopago.com`) está bloqueada por default en la política de red del entorno — hay que habilitarla en Network access de la configuración del entorno; (b) el checkout real (`www.mercadopago.com.ar/checkout/...`) rechaza con 403 el acceso automatizado/headless (anti-bot) — **no intentar sortear esto**, el pago se completa a mano en un navegador propio, logueado con una cuenta de prueba "Comprador" (nunca la misma cuenta "operador" de las credenciales, eso tira "una de las partes es de prueba"); (c) como `notification_url` apunta a `localhost:8081` (no accesible desde internet), el webhook real de MercadoPago nunca llega — hay que simularlo a mano pegándole directo a `POST /Pago/webhook` con `type=payment` y `data.id=<id real de la operación>` una vez completado el pago, lo cual dispara el mismo código (`procesarNotificacionPago`) que vuelve a consultar el estado real contra la API, no un mock.
3. **El flujo de alias de cobro (`POST /Usuario/alias-cobro/validar` y `/confirmar`) no tenía ninguna pantalla en el frontend** — detectado en la auditoría del Lote 4.1: el backend soporta el paso a paso completo (validar formato/banco sin guardar, luego confirmar y persistir) desde los lotes anteriores, pero ningún componente lo llamaba. Sin esto, un anfitrión real no tiene forma de indicarle a la plataforma a dónde pagarle (bloquea el Payout aunque funcionara). **Ya resuelto** — se agregó la sección "Alias de cobro" en `mi-perfil` (visible solo para `ANFITRION`/`ADMIN`, mismo patrón de `rol()` que el navbar), con el flujo de 2 pasos (`UsuarioService.validarAliasCobro`/`confirmarAliasCobro`) y el input sin atributo `name` (ver el bug de `NgModel` más abajo). Verificado en vivo con un CBU real contra el backend: identificó el banco (BBVA Argentina), mostró la advertencia de no poder confirmar el titular, y persistió el alias tras confirmar.